



JC4002 Knowledge Representation

Day 4: Prolog I

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 4

Prolog

1. Prolog
2. Syntax
3. Unification
4. Interpreter

Prolog

What is Prolog?

- A logic programming language
- A declarative language (with procedural interpretation)
- A knowledge base consisting of facts and rules
- Execution of queries over the KB
- Interpreter uses resolution to answer queries
- Based on Horn clauses (FOL is undecidable)

Prolog

What is Prolog?

- A logic programming language
- A declarative language (with procedural interpretation)
- A knowledge base consisting of facts and rules
- Execution of queries over the KB
- Interpreter uses resolution to answer queries
- Based on Horn clauses (FOL is undecidable)

Where can it be most useful?

- Expert systems
- Automated theorem proving
- Natural language processing

Day 4

Prolog

1. Prolog
- 2. Syntax**
3. Unification
4. Interpreter

Basic Syntax I

Everything is considered a **term**; there are different types of terms

- **Atoms**: sequence of characters beginning with lower case, or a sequence of characters enclosed in single quotes, or a sequence of special characters
 - ferdi
 - ferdian_jovan
 - 'Knowledge Representation and Reasoning'
 - @=>

Basic Syntax I

Everything is considered a **term**; there are different types of terms

❑ **Atoms:** sequence of characters beginning with lower case, or a sequence of characters enclosed in single quotes, or a sequence of special characters

- ferdi
- ferdian_jovan
- 'Knowledge Representation and Reasoning'
- @=>

❑ **Numbers:**

- 11
- 11.03

Basic Syntax I

Everything is considered a **term**; there are different types of terms

- ❑ **Atoms:** sequence of characters beginning with lower case, or a sequence of characters enclosed in single quotes, or a sequence of special characters
 - ferdi
 - ferdian_jovan
 - 'Knowledge Representation and Reasoning'
 - @=>
- ❑ **Numbers:**
 - 11
 - 11.03
- ❑ **Variables:** sequence of characters beginning with upper case or underscore
 - ❑ X
 - ❑ Rafael
 - ❑ _input
 - ❑ _ (special case, anonymous variable which lets Prolog assign a value independent of any other value)

Basic Syntax II

□ Compound terms:

- A functor followed by a sequence of arguments
- Functor must be an atom
- Arguments can be anything (including other compound terms)
- Arity represents the number of arguments of a compound term
- Examples
 - `module_lecturer('KRR', 'Ferd')`
 - `module_lecturer('KRR', 'Ferdian', 'Jovan')`
 - `module_assessments('KRR', [a1, a2, a3])`
- Common practice when referring to a specific predicate to use functor/arity
 - `module_lecturer/2`
 - `module_lecturer/3`
 - `module_assessments/2`
- Special cases
 - Strings: "Hello world!"
 - Lists: next slide

List

- ❑ Ordered set of terms
- ❑ Follow the structure of $[H \mid T]$, where H is the head of the list and T is the tail
- ❑ Examples of list
 - $[a, 1, [p(a), 2], b]$
 - `modules([mas, krr, ml])`
- ❑ Special list: $[\]$ the empty list
- ❑ How to manipulate lists? Tune in for the next lecture!

Syntax Examples

Match the right column with the left

- (A) Atom
- (N) Number
- (V) Variable
- (T) Compound Term
- (E) Error

- () University of Aberdeen
- () UoA
- () uoA
- () _uoA
- () _UoA
- () 'University of Aberdeen'
- () "University of Aberdeen"
- () university('UoA')
- () University(UoA)
- () 11.01
- () 11a
- () a11

Syntax Examples

Match the right column with the left

(A) Atom

(N) Number

(V) Variable

(T) Compound Term

(E) Error

(E) University of Aberdeen

() UoA

() uoA

() _uoA

() _UoA

() 'University of Aberdeen'

() "University of Aberdeen"

() university('UoA')

() University(UoA)

() 11.01

() 11a

() a11

Syntax Examples

Match the right column with the left

- (A) Atom
- (N) Number
- (V) Variable
- (T) Compound Term
- (E) Error

- (E) University of Aberdeen
- (V) UoA
- () uoA
- () _uoA
- () _UoA
- () 'University of Aberdeen'
- () "University of Aberdeen"
- () university('UoA')
- () University(UoA)
- () 11.01
- () 11a
- () a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
() _uoA
() _UoA
() 'University of Aberdeen'
() "University of Aberdeen"
() university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
() _UoA
() 'University of Aberdeen'
() "University of Aberdeen"
() university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
() 'University of Aberdeen'
() "University of Aberdeen"
() university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
() "University of Aberdeen"
() university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
(T) "University of Aberdeen"
() university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
(T) "University of Aberdeen"
(T) university('UoA')
() University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
(T) "University of Aberdeen"
(T) university('UoA')
(E) University(UoA)
() 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
(T) "University of Aberdeen"
(T) university('UoA')
(E) University(UoA)
(N) 11.01
() 11a
() a11

Syntax Examples

Match the right column with the left

(A) Atom

(N) Number

(V) Variable

(T) Compound Term

(E) Error

(E) University of Aberdeen

(V) UoA

(A) uoA

(V) _uoA

(V) _UoA

(A) 'University of Aberdeen'

(T) "University of Aberdeen"

(T) university('UoA')

(E) University(UoA)

(N) 11.01

(E) 11a

() a11

Syntax Examples

Match the right column with the left

(A) Atom
(N) Number
(V) Variable
(T) Compound Term
(E) Error

(E) University of Aberdeen
(V) UoA
(A) uoA
(V) _uoA
(V) _UoA
(A) 'University of Aberdeen'
(T) "University of Aberdeen"
(T) university('UoA')
(E) University(UoA)
(N) 11.01
(E) 11a
(A) a11

Negation

- ❑ Closed world assumption: what we cannot prove to be true, is false
- ❑ Achieved by **negation as failure**
- ❑ To prove not P, we try to prove P
- ❑ If proving P fails, then not P is true
- ❑ If we prove P, then not P fails
- ❑ Prolog operator for negation as failure is `\+ / 1` (functor `\+` with an arity of 1)

Negation Example

Considering the following KB:

- likes(alice,X) :- burger(X), \+ burger_king(b).
- burger(X) :- big_mac(X).
- burger(X) :- burger_king(X).
- big_mac(a).
- burger_king(b).

What would be the result for the query:

- ?- likes(alice, X).

Let's try to understand why using <https://swish.swi-prolog.org/>

Negation Example

Considering the following KB:

- likes(alice,X) :- burger(X), \+ burger_king(b).
- burger(X) :- big_mac(X).
- burger(X) :- burger_king(X).
- big_mac(a).
- burger_king(b).

What would be the result for the query:

- ?- likes(alice, X).

false

Let's try to understand why using <https://swish.swi-prolog.org/>

Activity 1

In pairs, discuss the following question (5 minutes) and then share your answers with the rest of your colleagues (5 minutes):

“What can we do to make sure Alice likes something?”

Day 4

Prolog

1. Prolog
2. Syntax
- 3. Unification**
4. Interpreter

Unification

- ❑ t_1 unifies with t_2 if there is a **substitution** S such that:

$$\text{apply}(S, t_1) = \text{apply}(S, t_2)$$

- ❑ A substitution is a function mapping variables to terms Consider the following unification attempt:

$$p(X, f(X), a) = p(Y, f(b), Y).$$

- ❑ Does this unify?

Unification

- ❑ t_1 unifies with t_2 if there is a **substitution** S such that:

$$\text{apply}(S, t_1) = \text{apply}(S, t_2)$$

- ❑ A substitution is a function mapping variables to terms Consider the following unification attempt:

$$p(X, f(X), a) = p(Y, f(b), Y).$$

- ❑ Does this unify? No!
- ❑ But why?

Unification

- ❑ t_1 unifies with t_2 if there is a **substitution** S such that:

$$\text{apply}(S, t1) = \text{apply}(S, t2)$$

- ❑ A substitution is a function mapping variables to terms Consider the following unification attempt:

$$p(X, f(X), a) = p(Y, f(b), Y).$$

- ❑ Does this unify? No!
- ❑ But why? $X = Y$, $X = b$, $Y = b$, $Y = a$
- ❑ Can we fix it?

Unification

- ❑ t_1 unifies with t_2 if there is a **substitution** S such that:

$$\text{apply}(S, t1) = \text{apply}(S, t2)$$

- ❑ A substitution is a function mapping variables to terms Consider the following unification attempt:

$$p(X, f(X), a) = p(Y, f(b), Y).$$

- ❑ Does this unify? No!
- ❑ But why? $X = Y$, $X = b$, $Y = b$, $Y = a$
- ❑ Can we fix it?

$$p(X, f(X), a) = p(Y, f(a), Y).$$

Unification Example

Do these terms unify? What are the substitutions?

- ☐ `data(11, Y, 2022) = data(X, 03, Z).`
- ☐ `day(03) = day('03').`
- ☐ `month(march) = month(March).`
- ☐ `year(2022) = year(_).`
- ☐ `"hello world" = 'hello world'.`
- ☐ `assessments([a1,a2,a3]) = assessments([H | T]).`

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

☐ $\text{year}(2022) = \text{year}(_).$

☐ $\text{"hello world"} = \text{'hello world'}.$

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

- false

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

☐ $\text{year}(2022) = \text{year}(_).$

☐ $\text{"hello world"} = \text{'hello world'}.$

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

- false

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

- $\text{March} = \text{march}$

☐ $\text{year}(2022) = \text{year}(_).$

☐ $\text{"hello world"} = \text{'hello world'}.$

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

- false

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

- $\text{March} = \text{march}$

☐ $\text{year}(2022) = \text{year}(_).$

- true

☐ $\text{"hello world"} = \text{'hello world'}.$

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

- false

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

- $\text{March} = \text{march}$

☐ $\text{year}(2022) = \text{year}(_).$

- true

☐ $\text{"hello world"} = \text{'hello world'}.$

- false

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

Unification Example

Do these terms unify? What are the substitutions?

☐ $\text{data}(11, Y, 2022) = \text{data}(X, 03, Z).$

- $X = 11, Y = 03, Z = 2022$

☐ $\text{day}(03) = \text{day}('03').$

- false

☐ $\text{month}(\text{march}) = \text{month}(\text{March}).$

- $\text{March} = \text{march}$

☐ $\text{year}(2022) = \text{year}(_).$

- true

☐ $\text{"hello world"} = \text{'hello world'}.$

- false

☐ $\text{assessments}([a1, a2, a3]) = \text{assessments}([H | T]).$

- $H = a1, T = [a2, a3]$

Day 4

Prolog

1. Prolog
2. Syntax
3. Unification
4. **Interpreter**

High-Level Interpreter Procedure

Input: a goal (query) G and a program P
Output: instances of G that are a logical consequence of P , or false otherwise
Algorithm: initialise the resolvent with G
 while resolvent is not empty **do**
 choose a goal A from the resolvent
 choose a clause from P such that $A \leftarrow B_1, \dots, B_n$
 and unify any variables with m.g.u. ϑ
 if no such clause exist **then** exit the loop,
 else replace A by $B_1, \dots, B_n$ in the
 resolvent, apply ϑ to the resolvent and to G
 if the resolvent is empty **then** output G
 else output false

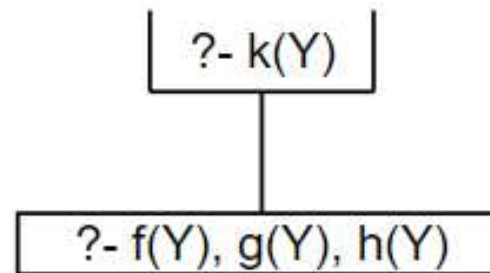
Proof Search

- ❑ In other words, the interpreter reads the KB and tries to unify the query with a fact or the head of a rule
- ❑ This procedure is done *topdown*
- ❑ Considering the following KB:
 - $f(a).$
 - $f(b).$
 - $g(a).$
 - $g(b).$
 - $h(b).$
 - $k(X) :- f(X), g(X), h(X).$
- ❑ What would be the result for the query:
 - $?- k(Y).$

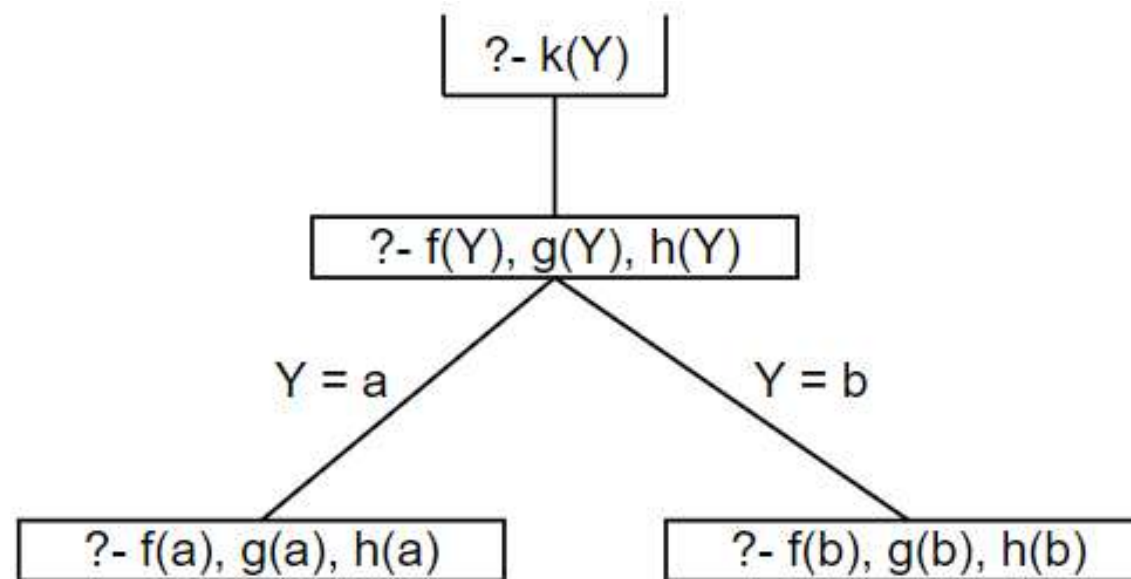
Search Tree

$\boxed{?-k(Y)}$

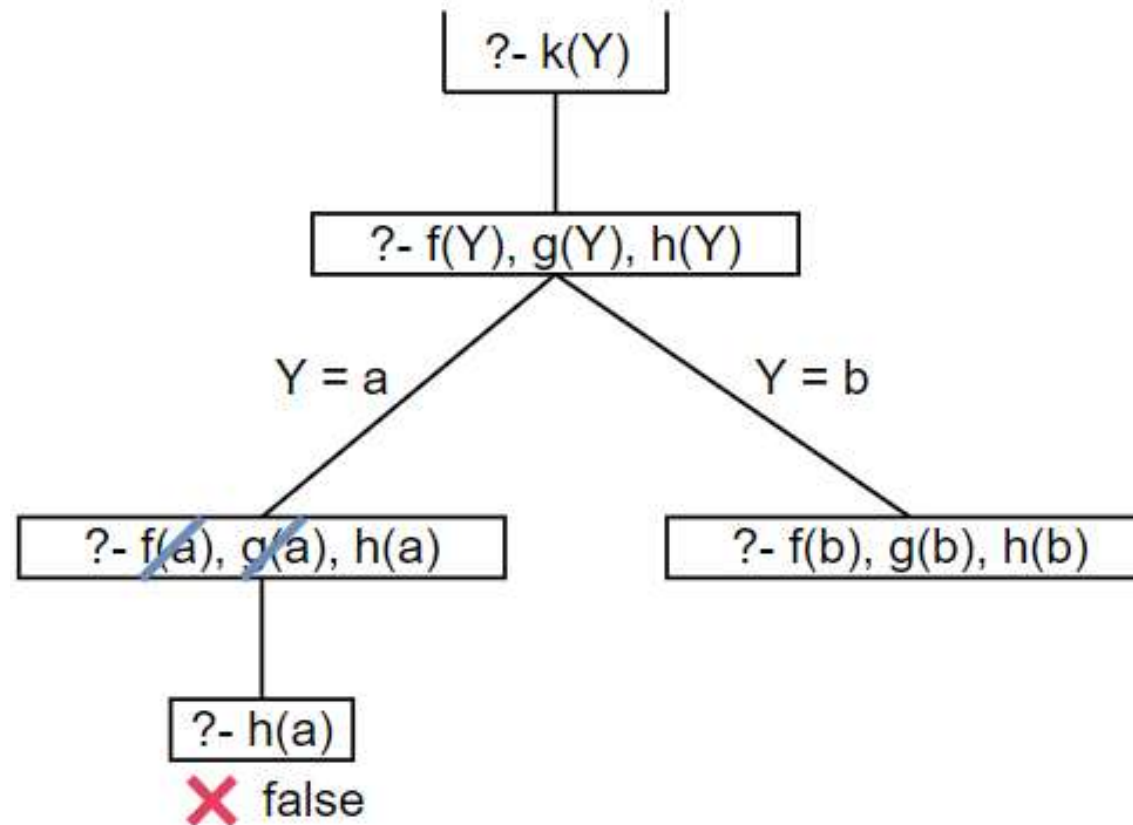
Search Tree



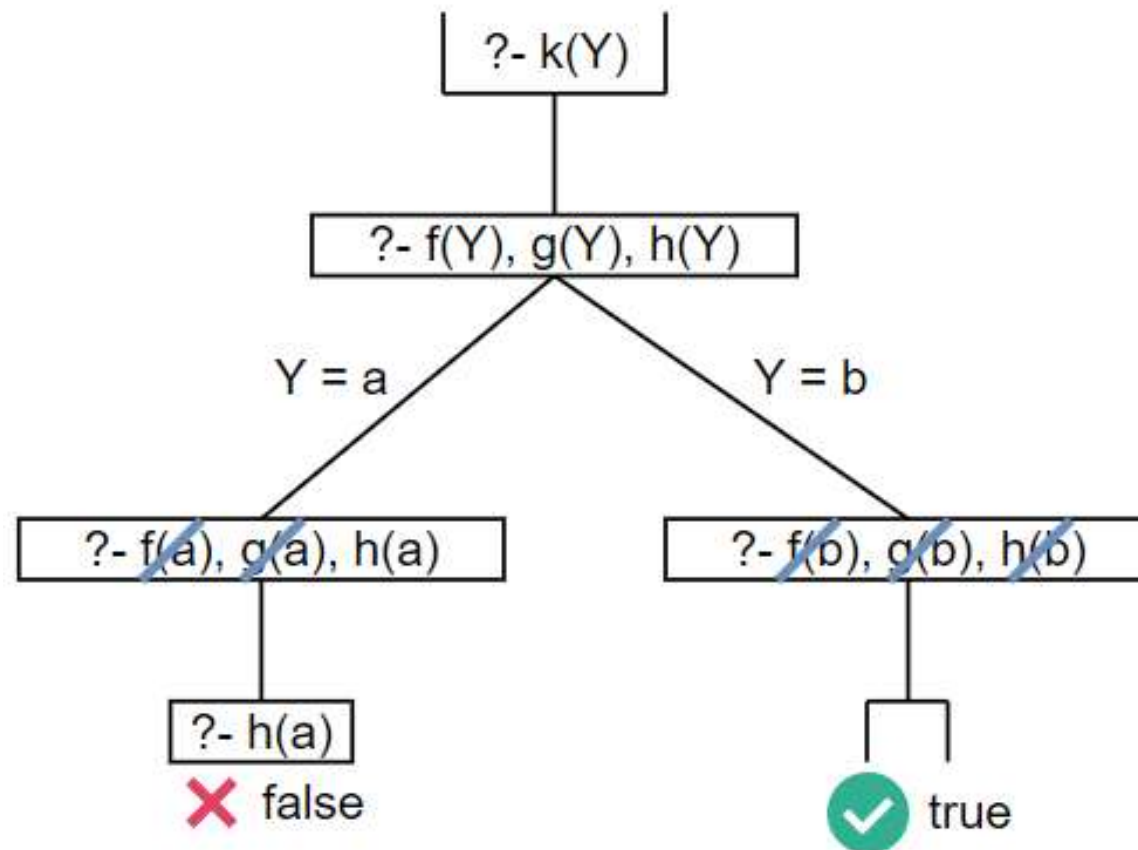
Search Tree



Search Tree



Search Tree



Summary

- Introduction to Prolog
- Basic syntax
- Lists and negation
- Unification
- Interpreter

Until the next session:

- Install SWI-Prolog to your computer.